



Fișă de lucru – Modificarea dinamică, Slicing și sortarea listelor în Python

Noțiuni teoretice :

1. **Mutabilitate:** Listele sunt structuri *mutabile*. Spre deosebire de șirurile de caractere (care nu pot fi modificate după creare), elementele unei liste pot fi schimbate, șterse sau adăugate direct în aceeași adresă de memorie, fără a se genera o listă nouă.

- *Exemplu:* Dacă avem lista `note = [8, 9, 10]` și executăm `note[0] = 7`, valoarea 8 este înlocuită pe loc, iar lista devine `[7, 9, 10]`.

2. **Eterogenitate:** Proprietatea listelor de a stoca simultan tipuri diferite de date în cadrul aceleiași structuri (numere întregi, numere reale, șiruri de caractere/text sau valori booleene).

- *Exemplu:* `date_produs = ["Laptop", 5, 2450.50, True]` (conține un text, un număr întreg, un număr real și o valoare booleană).

3. **Slicing (Segmentare):** Tehnica prin care se extrage o porțiune (o sublistă) dintr-o listă existentă, folosind sintaxa `lista[start:stop]`. Elementul de la poziția start este inclus, dar cel de la poziția stop este întotdeauna exclus.

- *Exemplu:* Dacă `numere = [10, 20, 30, 40, 50]`, atunci `numere[1:4]` va extrage elementele de la indecșii 1, 2 și 3, adică sublista `[20, 30, 40]`.

4. Metode fundamentale de modificare:

- **.remove(valoare)** — Caută și șterge prima apariție a valorii specificate din listă. Dacă valoarea nu există, va returna o eroare.
 - *Exemplu:* Pentru `culori = ["roșu", "albastru", "verde"]`, comanda `culori.remove("albastru")` transformă lista în `["roșu", "verde"]`.
- **.append(element)** — Adaugă un element nou la sfârșitul listei, măbind lungimea acesteia cu 1.
 - *Exemplu:* Pentru `fructe = ["măr", "banană"]`, comanda `fructe.append("cireșe")` transformă lista în `["măr", "banană", "cireșe"]`.
- **.insert(index, element)** — Inserează un element nou la o anumită poziție specificată prin index. Toate elementele aflate după această poziție sunt deplasate automat cu o poziție spre dreapta.
 - *Exemplu:* Pentru `orase = ["Cluj", "Brașov"]`, comanda `orase.insert(1, "Sibiu")` inserează textul pe poziția a doua, iar lista devine `["Cluj", "Sibiu", "Brașov"]`.
- **.pop()** — Elimină și returnează ultimul element dintr-o listă (dacă parantezele sunt goale). Poate primi și un index ca parametru (ex: `.pop(0)` șterge primul element).
 - *Exemplu:* Pentru `stoc = [10, 20, 30]`, comanda `ultimul = stoc.pop()` va lăsa lista sub forma `[10, 20]`, iar în variabila `ultimul` se va salva valoarea 30.
- **.sort()** — Reorganizează și ordonează elementele listei în ordine crescătoare (numerică sau alfabetică), modificând direct ordinea elementelor în memorie.
 - *Exemplu:* Pentru `valori = [45, 8, 23]`, comanda `valori.sort()` rearanjează elementele, iar lista devine `[8, 23, 45]`.

Activitate practica:

Scenariu: „Misiunea Administratorului de Sistem la o Agenție Spațială”

Context: Ești programatorul principal la centrul de control al zborurilor spațiale. Ai primit o listă de date care conține identificatorul text al misiunii și greutatea inițială a unor module satelitare exprimate în tone. Sistemul automat are nevoie ca această listă să fie curățată, actualizată dinamic și ordonată înainte de lansare.

✂ Cerințe :

Implementați în Python următoarele:

Pasul 1: Lista eterogenă

Declarați o listă numită `date_misiune` care conține elementele în următoarea ordine: textul "NASA_X1", numărul întreg 18, numărul real 14.5, numărul întreg 9 și numărul întreg 25.

Pasul 2: Eliminarea datelor de tip text

Pentru a evita erorile la calculele matematice, eliminați din listă identificatorul text "NASA_X1" folosind metoda potrivită bazată pe valoare. Afișați lista pe ecran pentru a verifica modificarea.

- *Cod sugestiv:* `date_misiune.remove(...)`

Pasul 3: Adăugarea unui nou modul (La sfârșit)

Un nou modul satelitar cu greutatea de 30 de tone a fost atașat la coada rachetei. Adăugați această valoare la sfârșitul listei folosind metoda specifică și afișați noua stare a listei.

- *Cod sugestiv:* `date_misiune.append(...)`

Pasul 4: Inserarea unui modul prioritar (La un index specific)

Inginerii au trimis un modul de urgență cu greutatea de 12 tone care trebuie poziționat neapărat pe **a doua poziție** a rachetei (adică la indexul 1). Utilizați metoda de inserare la poziție fixă și afișați lista pe ecran.

Pasul 5: Controlul prin feliere (Slicing)

Inspectorii de siguranță cer un raport rapid despre modulele aflate doar la indexul 1 și indexul 2. Extrageți această porțiune sub forma unei noi liste numite `sublista_control` folosind tehnica de feliere. Afișați pe ecran textul "Sublista extrasă:" urmat de valorile obținute.

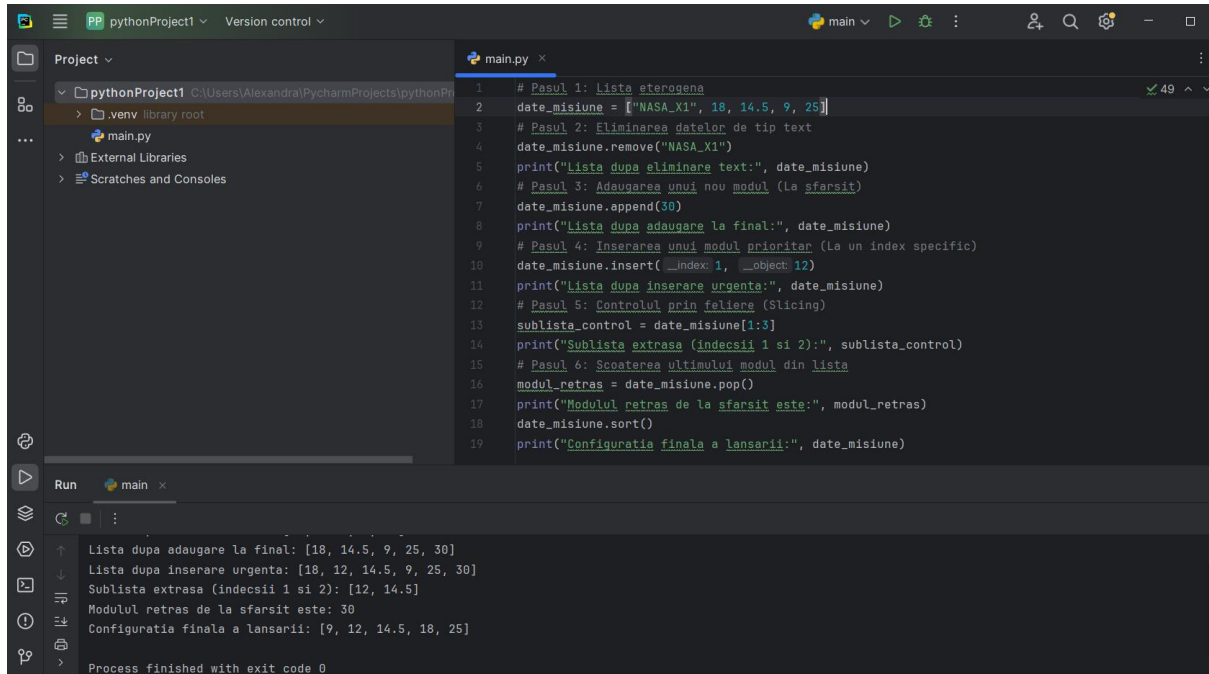
Pasul 6: Scoaterea ultimului modul din listă

Din cauza condițiilor meteo, ultimul modul adăugat trebuie dat jos de pe rachetă. Eliminați-l folosind metoda care șterge automat de la sfârșit și salvați valoarea eliminată într-o variabilă numită `modul_retras`. Afișați pe ecran valoarea salvată.

- *Cod sugestiv:* `modul_retras = date_misiune.pop()`

Pasul 7: Pregătirea finală (Ordonarea structurii)

Pentru stabilitatea rachetei în timpul zborului, modulele rămase trebuie ordonate în ordine crescătoare a greutateii lor. Aplicați metoda de sortare directă și afișați pe ecran configurația finală a listei cu mesajul "Configurația finală a lansării:".



The screenshot shows the PyCharm IDE with a Python project named 'pythonProject1'. The main.py file contains the following code:

```
1 # Pasul 1: Lista eterogena
2 date_misiune = ["NASA_X1", 18, 14.5, 9, 25]
3 # Pasul 2: Eliminarea datelor de tip text
4 date_misiune.remove("NASA_X1")
5 print("Lista dupa eliminare text:", date_misiune)
6 # Pasul 3: Adaugarea unui nou modul (la sfarsit)
7 date_misiune.append(30)
8 print("Lista dupa adaugare la final:", date_misiune)
9 # Pasul 4: Inserarea unui modul prioritar (la un index specific)
10 date_misiune.insert(_index=1, _object=12)
11 print("Lista dupa inserare urgenta:", date_misiune)
12 # Pasul 5: Controlul prin feliere (Slicing)
13 sublista_control = date_misiune[1:3]
14 print("Sublista extrasa (indecsii 1 si 2):", sublista_control)
15 # Pasul 6: Scoaterea ultimului modul din lista
16 modul_retras = date_misiune.pop()
17 print("Modulul retras de la sfarsit este:", modul_retras)
18 date_misiune.sort()
19 print("Configuratia finala a lansarii:", date_misiune)
```

The Run console shows the following output:

```
Lista dupa adaugare la final: [18, 14.5, 9, 25, 30]
Lista dupa inserare urgenta: [18, 12, 14.5, 9, 25, 30]
Sublista extrasa (indecsii 1 si 2): [12, 14.5]
Modulul retras de la sfarsit este: 30
Configuratia finala a lansarii: [9, 12, 14.5, 18, 25]
```

Process finished with exit code 0

Problemă rezolvată și explicată pas cu pas

Enunț: Se dă inițial o listă eterogenă numită `date_magazin`, care stochează numele unui brand și cantitățile de produse vândute: `date_magazin = ["Altex", 15, 45, 8, 23]`. Să se scrie un program în limbajul Python care realizează următoarele prelucrări succesive în memorie, demonstrând proprietățile de mutabilitate și eterogenitate ale listelor:

1. Elimină elementul de tip șir de caractere ("Altex") de la începutul colecției pentru ca lista să devină pur numerică.
2. Adaugă valoarea numerică 30 la sfârșitul listei.
3. Inserează valoarea numerică 12 pe a doua poziție în listă (la indexul 1).
4. Extrage și afișează o sublistă prin feliere (slicing) formată din elementele aflate între indexul 1 (inclusiv) și indexul 3 (exclusiv).
5. Elimină ultimul element din listă și îl afișează ca element salvat.
6. Sortează lista rămasă în ordine crescătoare și afișează structura finală.

3. Exemplu

- **Date de intrare (lista inițială):** `date_magazin = ["Altex", 15, 45, 8, 23]`
- **Date de ieșire (afișate pe ecran):**
 - Sublista extrasă: `[12, 45]`
 - Elementul eliminat: `30`
 - Lista finală sortată: `[8, 12, 15, 23, 45]`

4. Analiza exemplului

Pentru a înțelege cum se ajunge la rezultatele afișate, analizăm starea listei în memorie după fiecare instrucțiune obligatorie din enunț:

- **Starea inițială:** `date_magazin = ["Altex", 15, 45, 8, 23]` (Colecție eterogenă cu text și numere, lungime inițială 5).
- **Operația 1 (Eliminare text):** Se șterge elementul "Altex". Lista devine pur numerică: `[15, 45, 8, 23]`. Toate elementele rămase se reindexează automat la stânga începând cu poziția 0.
- **Operația 2 (Adăugare la sfârșit):** Se adaugă valoarea 30 la final. Lista devine: `[15, 45, 8, 23, 30]`.
- **Operația 3 (Inserare pe poziție):** Se introduce numărul 12 la indexul 1 (a doua poziție). Elementele de după el se deplasează la dreapta. Lista devine: `[15, 12, 45, 8, 23, 30]`.
- **Operația 4 (Feliere / Slicing):** Se extrag elementele de la indexul 1 până la indexul 3 (fără a include indexul 3). Indexul 1 este 12, indexul 2 este 45. Se salvează și se afișează sublista: `[12, 45]`.
- **Operația 5 (Eliminare de la sfârșit):** Se extrage ultimul element din listă, adică valoarea 30. Lista redevine: `[15, 12, 45, 8, 23]`. Valoarea afișată ca fiind eliminată este 30.
- **Operația 6 (Sortare crescătoare):** Lista se ordonează direct în memorie de la cel mai mic la cel mai mare element. Lista finală sortată devine: `[8, 12, 15, 23, 45]`.

Pseudocod:

```
date_magazin ← ["Altex", 15, 45, 8, 23] // Inițializarea listei eterogene în memorie
```

```
șterge_valoare(date_magazin, "Altex") // Pasul 1: Eliminarea elementului text
```

```
adaugă_la_sfârșit(date_magazin, 30) // Pasul 2: Adăugarea unei noi valori la sfârșit
```

```
inserează_la_poziție(date_magazin, 1, 12) // Pasul 3: Inserarea unei valori la indexul 1
```

```
sublista ← extrage_sublistă(date_magazin, 1, 3) // Pasul 4: Felierea (Slicing) de la indexul 1 la 3 (exclusiv)
```

```
scrie "Sublista extrasă (indecsii 1 și 2):", sublista
```

```
element_sters ← șterge_și_returnează_ultimul(date_magazin) // Pasul 5: Metoda Pop
```

```
scrie "Elementul eliminat de la sfârșit este:", element_sters
```

```
sortează_crescător(date_magazin) // Pasul 6: Sortarea listei rămase în ordine crescătoare
```

```
scrie "Lista finală sortată este:", date_magazin
```

Implementarea în limbajul Python :

```
date_magazin = ["Altex", 15, 45, 8, 23]
date_magazin.remove("Altex")
date_magazin.append(30)
date_magazin.insert(1, 12)
sublista = date_magazin[1:3]
print("Sublista extrasă (indecsii 1 și 2):", sublista)
element_sters = date_magazin.pop()
print("Elementul eliminat de la sfârșit este:", element_sters)
date_magazin.sort()
print("Lista finală sortată este:", date_magazin)
```

Testarea și verificarea pas cu pas

Pasul (Instrucțiunea executată)	Starea listei date_magazin în memorie	Indecșii pozitivi curepți	Variabile create / Rezultate
Stare inițială	["Altex", 15, 45, 8, 23]	0: "Altex", 1: 15, 2: 45, 3: 8, 4: 23	Alocare listă eterogenă
1. .remove("Altex")	[15, 45, 8, 23]	0: 15, 1: 45, 2: 8, 3: 23	Textul a fost eliminat din memorie
2. .append(30)	[15, 45, 8, 23, 30]	0: 15, 1: 45, 2: 8, 3: 23, 4: 30	30 adăugat la sfârșitul structurii
3. .insert(1, 12)	[15, 12, 45, 8, 23, 30]	0: 15, 1: 12, 2: 45, 3: 8, 4: 23, 5: 30	12 inserat dinamic la indexul 1
4. [1:3] (Feliere)	[15, 12, 45, 8, 23, 30]	(Nemodificată structural)	sublista = [12, 45] (indecșii 1 și 2)
5. .pop()	[15, 12, 45, 8, 23]	0: 15, 1: 12, 2: 45, 3: 8, 4: 23	element_sters = 30
6. .sort()	[8, 12, 15, 23, 45]	0: 8, 1: 12, 2: 15, 3: 23, 4: 45	Rearanjare completă crescătoare

Afișare pe ecran:

Sublista extrasă (indecșii 1 și 2): [12, 45]

Elementul eliminat de la sfârșit este: 30

Lista finală sortată este: [8, 12, 15, 23, 45]

Problemă propusă pentru tema de acasă

🎵 Scenariu: „Aplicația de Streaming (Gestionarea unui Playlist)”

Context: Ești dezvoltatorul unei aplicații moderne de muzică (similară cu Spotify). Primești din baza de date o listă eterogenă brută numită `playlist_utilizator`, care conține numele utilizatorului premium și numărul de redări (vizualizări exprimate în milioane) ale melodiilor din lista sa de favorite. Pentru ca algoritmul să ruleze corect, lista trebuie modificată dinamic în memorie.

Enunț:

Se dă inițial lista: `playlist_utilizator = ["Andrei_Music", 18, 42, 5, 29]`. Scrieți un program în limbajul Python care parcurge următoarele cerințe obligatorii (fiecare pas modifică direct lista rezultată de la pasul anterior):

1. **Curățarea datelor:** Eliminați din listă numele utilizatorului ("Andrei_Music") folosind metoda bazată pe valoare, astfel încât în colecție să rămână doar date pur numerice.
2. **Actualizare rapidă:** Utilizatorul apasă butonul „Inimioară” (Favorite) pe o nouă melodie care are 35 de milioane de redări. Adăugați această valoare la sfârșitul listei.
3. **Piesă recomandată (Prioritate):** Algoritmul aplicației introduce o melodie extrem de populară cu 14 milioane de redări direct pe **a doua poziție** din listă (adică la indexul 1). Utilizați metoda potrivită pentru inserarea la index specific.
4. **Raport de feliere (Slicing):** Serverul cere o verificare rapidă a melodiilor intermediare. Extrageți prin feliere o sublistă numită `sublista_top` formată din elementele aflate între indexul 1 (inclusiv) și indexul 3 (exclusiv). Afișați sublista pe ecran cu mesajul "Melodii verificate la feliere:".
5. **Butonul „Skip / Șterge”:** Utilizatorul decide să ștergă ultima melodie adăugată în coada playlist-ului. Eliminați ultimul element din listă și salvați-l într-o variabilă numită `piesa_stearsa`. Afișați pe ecran valoarea extrasă cu mesajul "Numărul de redări al piesei eliminate:".
6. **Ordonare inteligentă:** Pentru o redare cursivă, sortați melodiile rămase în ordine crescătoare a vizualizărilor și afișați pe ecran structura finală a listei cu mesajul "Configurația finală a playlist-ului sortat:".